

부산대학교
PUSAN NATIONAL UNIVERSITY

Abstract Data Types

# Data Abstraction

Values, Operations, and Semantics

**DS3600082-140**
자료구조
2026학년도 2학기

**오용경**
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Section 1 • Abstract Data Types

# Data Abstraction

값과 연산을 정의하는 일과 실제 저장하는 일을 구분한다

# 자료형은 값과 연산으로 정한다

정의

## 자료형(data type)

가능한 값의 집합과 그 값에 적용할 수 있는 연산의 묶음이다.
연산(operation)은 값을 조회하거나 계산하고 변경하는 동작이다.

| 자료형 | 연산의 예 | 결과 |
|---|---|---|
| 정수 | `3 + 2` | `5` |
| 문자열 | `"3" + "2"` | `"32"` |

Adapted from Miller and Ranum (2013), Introduction.

# 추상 자료형이 정하는 것

정의

## 추상 자료형(abstract data type, ADT)

값과 연산의 의미로 정의하고, 저장 방식은 정하지 않은 자료형이다.
연산의 의미(semantics)는 호출의 반환값과 상태 변화를 뜻한다.

| | |
|---|---|
| **값** | 어떤 상태를 허용하는가 |
| **연산** | 어떤 동작을 요청하는가 |
| **연산의 의미** | 무엇을 반환하고 바꾸는가 |

Source: Morin (2013), Section 1.2.

# 데이터 추상화

정의

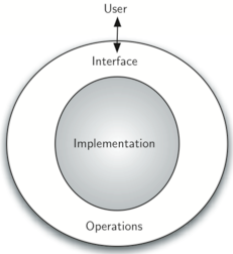
## 데이터 추상화(data abstraction)

연산의 사용 방법과 내부 처리 방법을 분리하는 원리다.
사용자는 연산의 의미를 알면 내부 저장 방식을 몰라도 자료를 다룰 수 있다.

| 사용자가 알아야 하는 것 | 구현이 결정하는 것 |
| --- | --- |
| `append(30)`은 끝에 30을 추가한다<br>기존 원소의 순서는 유지된다 | 새 원소를 저장할 공간을 확보하는 방법<br>공간이 부족할 때 기존 원소를 옮기는 방법 |

Adapted from Miller and Ranum (2013), Introduction.

# 추상화 경계 밖에 공개되는 정보




**바깥 고리인 인터페이스가 공개하는 것**

- 사용자는 인터페이스(interface)의 연산 이름과 입력을 안다
- 반환값과 상태 변화를 관찰한다
- 저장 공간을 관리하는 코드는 알 필요가 없다

Source: Miller and Ranum (2013), Figure 2, Abstract Data Type.

# 요구사항에서 ADT를 고르고 구현에서 연산 비용을 따진다




ADT는 필요한 결과를 정하며 구현에 따라 그 결과를 만드는 작업량이 달라진다

Checkpoint

# Data Abstraction

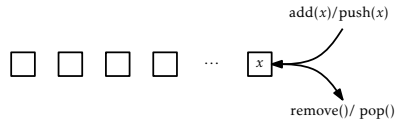
- 정수의 +와 문자열의 +는 결과가 어떻게 다른가?
- ADT를 정의할 때 저장 방식을 정해야 하는가?
- `append()`를 쓰기 위해 알아야 하는 것과 몰라도 되는 것은 무엇인가?

Section 2 • Abstract Data Types

# Operations and Semantics

같은 값을 담아도 ADT의 연산 규칙에 따라 반환값이 달라진다

# 스택의 연산과 상태 변화




스택(stack)은 `push(x)`로 *x*를 맨 위(그림 오른쪽 끝)에 추가하고 `pop()`으로 맨 위 원소를 삭제해 반환

**시작**

```
[]
```

**추가**

```
push(4)
[4]
```

**추가**

```
push("dog")
[4, "dog"]
```

**삭제**

```
pop()
반환 "dog"
남은 상태 [4]
```

Source: Morin (2013), Figure 1.3.

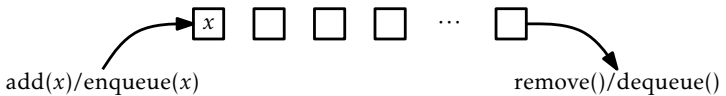
# 최근 방문 기록을 스택으로 되돌린다

| 단계 | 동작 | 방문 기록 | pop() 결과 |
|---|---|---|---|
| 0 | 부산대 페이지 방문 | [PNU] | 없음 |
| 1 | 강의 페이지 방문 | [PNU, Lecture] | 없음 |
| 2 | 과제 페이지 방문 | [PNU, Lecture, HW] | 없음 |
| 3 | 뒤로 가기 | [PNU, Lecture] | HW |
| 4 | 뒤로 가기 | [PNU] | Lecture |

가장 최근에 추가한 기록을 먼저 삭제해야 하면 스택의 연산 의미가 맞는다

Adapted from Miller and Ranum (2013), The Stack Abstract Data Type.

# 큐의 연산과 상태 변화




큐(queue)는 `enqueue(x)`로 뒤쪽(그림 왼쪽)에 추가하고 `dequeue()`로 앞쪽(그림 오른쪽) 원소를 삭제해 반환(아래는 앞쪽을 왼쪽에 표기)

**시작**

```
[]
```

→

**추가**

```
enqueue(4)
[4]
```

→

**추가**

```
enqueue("dog")
[4, "dog"]
```

→

**삭제**

```
dequeue()
반환 4
남은 상태
["dog"]
```

Source: Morin (2013), Figure 1.1.

# 접수한 인쇄 작업을 큐로 처리한다

| 단계 | 동작 | 대기 작업 | `dequeue()` 결과 |
|---|---|---|---|
| 0 | 보고서 접수 | `[report]` | 없음 |
| 1 | 과제 접수 | `[report, homework]` | 없음 |
| 2 | 포스터 접수 | `[report, homework, poster]` | 없음 |
| 3 | 한 건 인쇄 | `[homework, poster]` | `report` |
| 4 | 한 건 인쇄 | `[poster]` | `homework` |

먼저 접수한 작업을 먼저 처리해야 하면 큐의 연산 의미가 맞는다

Adapted from Miller and Ranum (2013), The Queue Abstract Data Type.

# 같은 추가 순서도 삭제 결과는 다르다

**추가한 값이 같아도 어떤 ADT로 해석하는지에 따라 먼저 나오는 값이 달라진다**

| ADT | 추가 순서 | 삭제 연산 | 반환값 |
|---|---|---|---|
| 스택 | `4, "dog", True` | `pop()` | `True` |
| 큐 | `4, "dog", True` | `dequeue()` | `4` |

> 최근 기록을 먼저 꺼내려면 스택, 먼저 접수한 값을 먼저 꺼내려면 큐를 선택한다

Adapted from Miller and Ranum (2013), Stack and Queue Abstract Data Types.

# 덱의 연산 집합

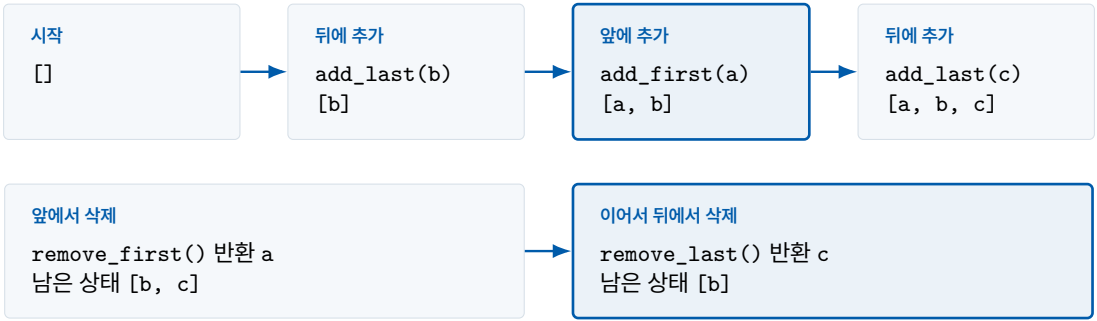
## 덱(deque)은 앞쪽과 뒤쪽에서 모두 값을 추가하거나 삭제할 수 있다

| 위치 | 추가 연산 | 삭제 연산 |
|---|---|---|
| 앞쪽 | `add_first(x)` | `remove_first()` |
| 뒤쪽 | `add_last(x)` | `remove_last()` |

> 같은 끝을 쓰면 스택, 서로 다른 끝을 쓰면 큐와 같은 삭제 순서를 만든다

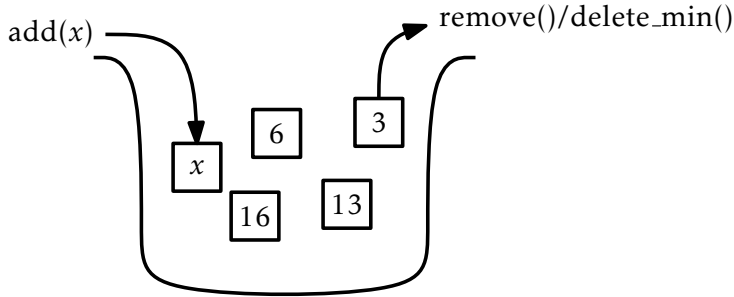
Source: Morin (2013), Section 1.2.1.

# 덱의 양쪽 끝을 추적한다




Adapted from Morin (2013), Section 1.2.1.

# 우선순위 큐의 삭제 기준




우선순위 큐(priority queue)는 값의 우선순위에 따라 반환할 원소를 결정하며, 그림에서는 가장 작은 값을 먼저 반환

Source: Morin (2013), Figure 1.2.

# 우선순위가 접수 순서보다 앞선다

| 접수 | 작업 | 우선순위 | 삭제 순서 |
|---|---|---|---|
| 1 | 일반 인쇄 | 3 | 세 번째 |
| 2 | 시험지 인쇄 | 1 | 첫 번째 |
| 3 | 강의자료 인쇄 | 2 | 두 번째 |

**접수**

일반 인쇄가 먼저

→

**비교**

숫자가 작을수록 우선

→

**삭제**

시험지를 먼저 반환

Adapted from Morin (2013), Section 1.2.1.

Checkpoint

# Operations and Semantics

- 같은 값을 스택과 큐에 넣었을 때 먼저 삭제되는 값은 각각 무엇인가
- 덱을 스택처럼 쓰려면 어느 끝에서 추가하고 삭제해야 하는가
- 저장된 값이 같아도 스택과 큐가 서로 다른 ADT인 이유는 무엇인가

Section 3 • Abstract Data Types

# ADT Specifications

값, 연산, 연산의 의미로 ADT가 정해진다

# ADT 명세의 구성

정의

## 명세(specification)

ADT가 허용하는 값과 연산을 정하고, 각 연산의 입력·반환값·적용 조건·상태 변화를 정확히 적은 것이다.

| | |
|---|---|
| **값의 집합** | 순서가 있는 원소의 유한 나열, 마지막 원소가 맨 위 |
| **연산** | `push(x)`, `pop()` |
| **의미와 적용 조건** | `push(x)`는 맨 위에 추가,<br>`pop()`은 맨 위 원소를 삭제해 반환(빈 상태면 호출 불가) |

Adapted from Morin (2013), Section 1.2.

# 연산을 상태 전이로 읽기




| $S$ | 연산 | $r$ | $S'$ |
|---|---|---|---|
| `[4, "dog"]` | `push(True)` | 없음 | `[4, "dog", True]` |
| `[4, "dog", True]` | `pop()` | `True` | `[4, "dog"]` |

Adapted from Morin (2013), Section 1.2.1.

# 상태를 읽는 연산과 바꾸는 연산

| 연산 | 반환값 | 호출 뒤 상태 | 분류 |
|---|---|---|---|
| `peek()` | 맨 위 원소 | 그대로 유지 | 조회 |
| `size()` | 원소 수 | 그대로 유지 | 조회 |
| `is_empty()` | `True` 또는 `False` | 그대로 유지 | 조회 |
| `push(x)` | 없음 | *x*가 맨 위에 추가 | 상태 변경 |
| `pop()` | 맨 위 원소 | 맨 위 원소 삭제 | 상태 변경 |

## 같은 반환값, 다른 호출 후 상태

| 호출 | 호출 전 | 반환값 | 호출 후 |
|---|---|---|---|
| `peek()` | `[4, "dog"]` | `"dog"` | `[4, "dog"]` |
| `pop()` | `[4, "dog"]` | `"dog"` | `[4]` |

Adapted from Miller and Ranum (2013), The Stack Abstract Data Type.

# 빈 스택의 삭제와 적용 조건

| 상태 | 호출 | 명세에서 확인할 것 |
|---|---|---|
| `[4]` | `pop()` | `4`를 반환하고 빈 상태가 됨 |
| `[]` | `pop()` | 적용 조건을 만족하지 않음 |
| `[]` | `is_empty()` | `True`를 반환하고 상태 유지 |

**Caution** 빈 상태의 `pop()`이 어떤 오류를 내는지는 구현이 정하더라도, 호출 가능한 조건은 명세에 적어야 한다

Adapted from Miller and Ranum (2013), The Stack Abstract Data Type.

# 연산 열을 따라 상태와 반환값을 계산한다

## 각 행의 호출 뒤 상태가 다음 행의 호출 전 상태가 된다

| 호출 전 상태 | 연산 | 반환값 | 호출 뒤 상태 |
|---|---|---|---|
| `[]` | `push(4)` | 없음 | `[4]` |
| `[4]` | `push(7)` | 없음 | `[4, 7]` |
| `[4, 7]` | `peek()` | `7` | `[4, 7]` |
| `[4, 7]` | `pop()` | `7` | `[4]` |

같은 상태에서 호출해 반환값이 같아도 호출 뒤 상태가 다르면 다른 연산이다

Adapted from Miller and Ranum (2013), The Stack Abstract Data Type.

Checkpoint

# ADT Specifications

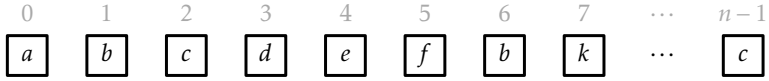
- `peek()`와 `pop()`은 반환값과 상태 변화가 어떻게 다른가
- 빈 스택에서 `pop()`을 호출할 수 있는지는 명세의 어느 항목에서 정하는가
- 빈 스택에 `push(4)`와 `pop()`을 차례로 실행하면 반환값과 남는 상태는 무엇인가?

Section 4 • Abstract Data Types

# Choosing an ADT

요구하는 관찰과 반환값으로 적합한 ADT를 구분한다

# 리스트의 위치와 상태 변화




인덱스는 0부터 시작하며 `get(i)`는 위치 *i*의 값을, `set(i, x)`는 교체 전 값을 반환. 아래 표는 길이 3인 리스트로 추적

| 단계 | 연산 | 리스트 상태 | 반환값 |
|---|---|---|---|
| 0 | 시작 | `[a, b, c]` | – |
| 1 | `get(2)` | `[a, b, c]` | `c` |
| 2 | `set(1, x)` | `[a, x, c]` | `b` |
| 3 | `add(1, y)` | `[a, y, x, c]` | 없음 |
| 4 | `remove(2)` | `[a, y, c]` | `x` |

Adapted from Morin (2013), Figure 1.4 and Section 1.2.2.

# 리스트 연산 하나가 바꾸는 것을 찾는다

| 호출 전 | 연산 | 반환값 | 호출 뒤 |
|---|---|---|---|
| [a, b, c] | get(1) | b | [a, b, c] |
| [a, b, c] | set(1, x) | b | [a, x, c] |
| [a, b, c] | add(1, x) | 없음 | [a, x, b, c] |
| [a, b, c] | remove(1) | b | [a, c] |

set(i, x)는 교체 전 값을 반환한다.



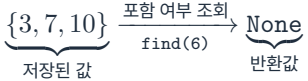

Adapted from Morin (2013), Section 1.2.2.

# USet의 포함 여부 연산

정의

## USet

중복 없는 값의 집합에 `add(x)`, `remove(x)`, `find(x)`를 제공하는 ADT다.
`find(x)`는 $x$와 같은 값이 있으면 그 값을, 없으면 `None`을 반환한다.




Source: Morin (2013), Section 1.2.3.

# SSet의 순서 연산

정의

## SSet

값에 순서가 있는 집합이며, USet과 연산은 같고 `find(x)`의 의미만 다른 ADT다.
`find(x)`는 $x$ 이상인 값 가운데 가장 작은 값을, 없으면 `None`을 반환한다.

$$\underbrace{\{3, 7, 10\}}_{\text{저장된 값}} \xrightarrow[\texttt{find(6)}]{\text{순서 조회}} \underbrace{7}_{\text{반환값}}$$

Source: Morin (2013), Section 1.2.4.

# 포함 여부와 순서 조회는 필요한 답이 다르다

**USet은 중복 없는 값의 집합, SSet은 순서가 있는 집합이다. $\{3, 7, 10\}$을 저장해도 `find`의 의미가 다르다**

| 질문 | ADT | 호출 | 반환값 |
|---|---|---|---|
| 6이 들어 있는가 | USet | `find(6)` | `None` |
| 6 이상인 최솟값은 무엇인가 | SSet | `find(6)` | 7 |
| 7이 들어 있는가 | USet | `find(7)` | 7 |

같은 이름의 연산도 ADT 명세에 따라 반환값의 의미가 달라진다

Adapted from Morin (2013), Sections 1.2.3 and 1.2.4.

# 요구사항에서 필요한 ADT 연산을 찾는다

| 자연어 요구 | 필요한 관찰 | 후보 연산 |
| --- | --- | --- |
| 최근 작업을 취소한다 | 최근에 추가한 원소를 삭제·반환 | 스택 `pop()` |
| 먼저 접수한 요청을 처리한다 | 첫 원소를 삭제·반환 | 큐 `dequeue()` |
| 긴급한 요청을 먼저 처리한다 | 우선순위가 가장 높은 원소 삭제 | 우선순위 큐 `remove()` |
| 특정 번호가 등록되었는지 확인한다 | 같은 값 또는 없음 | USet `find(x)` |
| 기준 시각 이후 첫 일정(같은 시각 포함) | 기준 이상 최솟값 또는 없음 | SSet `find(x)` |

> ADT를 고르기 전에 필요한 반환값, 삭제 위치, 순서 조건을 문장으로 분리한다

# Python 코드로 삭제 순서를 확인한다

```python
from collections import deque

items = deque([4, "dog", True])
print(items.pop())      # True
print(items.popleft())  # 4
```

| 호출 | 반환값 | 남은 원소 |
|---|---|---|
| `items.pop()` | `True` | `[4, "dog"]` |
| `items.popleft()` | `4` | `["dog"]` |

값을 뒤에 넣은 deque에서 뒤로 꺼내면 스택, 앞으로 꺼내면 큐의 순서다. 쓰는 연산이 ADT를 정한다

# Python 컬렉션의 조회 기준

```python
seq = [4, "dog", True]
print(seq[1])                  # dog
record = {"id": 4, "name": "dog"}
print("id" in record)          # True
print("dog" in record)         # False
tags = {4, "dog", True}
print("dog" in tags)           # True
```

| 코드 | 확인하는 것 |
|---|---|
| `seq[1]` | 두 번째 위치에 저장된 값 |
| `"id" in record` | 사전에 해당 키가 있는가 |
| `"dog" in tags` | 집합에 해당 값이 있는가 |

Adapted from Miller and Ranum (2013), Getting Started with Data.

Checkpoint

# Choosing an ADT

- 가장 최근 원소를 삭제해야 하는 문제에는 어떤 연산이 필요한가
- 포함 여부만 확인하는 문제와 인덱스로 조회하는 문제에는 각각 어떤 ADT가 필요한가
- Python 자료형 이름만 보고 ADT를 단정할 수 없는 이유는 무엇인가

# What Defines an Abstract Data Type?

**Next Lecture: Data Structure Interfaces**

- 인터페이스가 공개하는 연산 명세
- 같은 인터페이스를 만족하는 서로 다른 표현
- 클라이언트 코드와 구현의 분리


오용경 yongkyung.oh@pusan.ac.kr